DUNDALK INSTITUTE OF TECHNOLOGY

DK_ICSOP_7 - B.Sc. in Computing Systems and Operations

DK_ICCSO_8 – B.Sc. (Hons) in Computing Systems and Operations

Stage 3 - Semester 5

January 2025

COMP I8062 - Algorithms

Internal Examiner(s): Dr. Michelle Graham

External Examiner(s): xxxxxxx

Date:

Time:

Duration:

Venue:

No. Pages:

Instructions

Answer QUESTION ONE and any other TWO questions.

PLEASE NOTE: Exam Paper printed DOUBLE SIDED.

DO NOT TURN THIS PAGE UNTIL INSTRUCTED

You MUST attempt this question.

Question 1) Algorithm Design and Complexity

(a) Explain the following algorithm design paradigms. For each paradigm, illustrate your answer with a suitable example of an algorithm you have studied.

(i) Divide and conquer.
(ii) Recursive.
(iii) Brute force.

(12 marks)

(b) Examine the algorithm found in **appendix A**. It has been designed to process bell-shaped data, i.e. data that ascends to a peak, then decreases.

(i) Assume the algorithm is run with the following array as its input:
data = [-2, 1, 3, 15, 34, 56, 67, 55, 43, 33, 21, 19, 10, 11, 8, 3, -10]
What is the result of this algorithm? Explain how this result is reached.

(8 marks)

(ii) What is the paradigm being used for the algorithm's design? Explain the time and space complexity of the algorithm, justifying your answer with reference to specific lines of code.

(6 marks)

(iii) Write and explain pseudocode for an alternative (but still functional) solution to this problem. Is your version more, less or equally as efficient (for both time and space) as the original implementation? Justify your answer.

(14 marks)

Choose any TWO questions from questions 2, 3 and 4

Question 2) Sorting Algorithms

(a) What is meant by stability in sorting? Give an example where stability is required.

(5 marks)

(b) Compare and contrast **Merge sort** vs **Quick sort** under the following headings:

(i) Time complexity
(ii) Space complexity
(iii) Stability

(15 marks)

(c) Consider the following array: **[10, 9, 5, 7, 8]**
Apply the **optimised Bubble Sort** algorithm to sort this array into ascending order. Show the array after each step of the sorting process until it is fully sorted.

(10 marks)

Question 3) Recursive Algorithms

(a) The code shown in **appendix B** is a recursive algorithm checking if a supplied piece of text is a palindrome (a word that is spelled the same reading from left to right as it is reading from right to left).

(i) Explain the three essential components required for a recursive function to work correctly. For each component, describe why it is necessary.

(6 marks)

(ii) Identify and explain how each of the three essential components of recursion is implemented in the algorithm shown in **appendix B**.

(6 marks)

(iii) Explain the time and space complexity of the algorithm, justifying your answer with reference to specific lines of code.

(6 marks)

(b) Recursive algorithms can be optimised through implementing memoization.

(i) What is memoization? How can it optimise recursive algorithms?

(4 marks)

(ii) Rewrite the code of the original recursive algorithm above to include memoization. Does this change the space and time complexities you calculated in part 3a)? Explain your answer.

(8 marks)

Question 4) Selecting Algorithms

(a) Your team has been tasked with creating a MusicPlayer application. You have been assigned to work on the application's playlist functionality and need to manage a queue of Song objects stored as an array. The Songs are to be stored in order of preference, i.e. the Song with the highest rating should always appear first in the queue. The application is envisioned to work on a device with low power availability, therefore power conservation has been deemed of equal importance to response time.

(i) Outline two approaches to *array insertion* suitable for use in this context. Ensure that you have adapted them to maintain the sorted nature of the data **without** the need to sort the data after insertion.

(16 marks)

(ii) An alternative to the approaches you outlined in part (i) is to insert the new Song into the first available space after the data, then sort the resulting array. Write a short note comparing the performance of this approach to that of the approaches you outlined in (i).

(8 marks)

(iii) Which of the approaches you have discussed would you recommend for use in the scenario above, and why? Justify your answer with evidence based on the manner of operation of the algorithms, and any relevant details from the given scenario.

(6 marks)

APPENDIX A:

```
4    @    public static int mysteryAlgA(int [] data) {
5             int posA = 0;
6             int posB = data.length-1;
7
8             while(posA < posB) {
9                 int index = posA + (posB - posA)/2;
10                if(data[index] < data[index + 1]) {
11                    posA = index + 1;
12                }else {
13                    posB = index;
14                }
15            }
16            return data[posA];
17        }
```

APPENDIX B:

```
38  @    public static boolean isPalindrome(String text){
39           if(text.isEmpty()){
40               return true;
41           }
42           return isPal(text, start: 0, end: text.length()-1);
43       }
44
45       private static boolean isPal(String text, int start, int end){
46           if(start == end){
47               return true;
48           }
49
50           if(text.charAt(start) != text.charAt(end)){
51               return false;
52           }
53
54           if(start + 1 < end){
55               isPal(text, start: start+1, end: end-1);
56           }
57           return true;
58       }
```